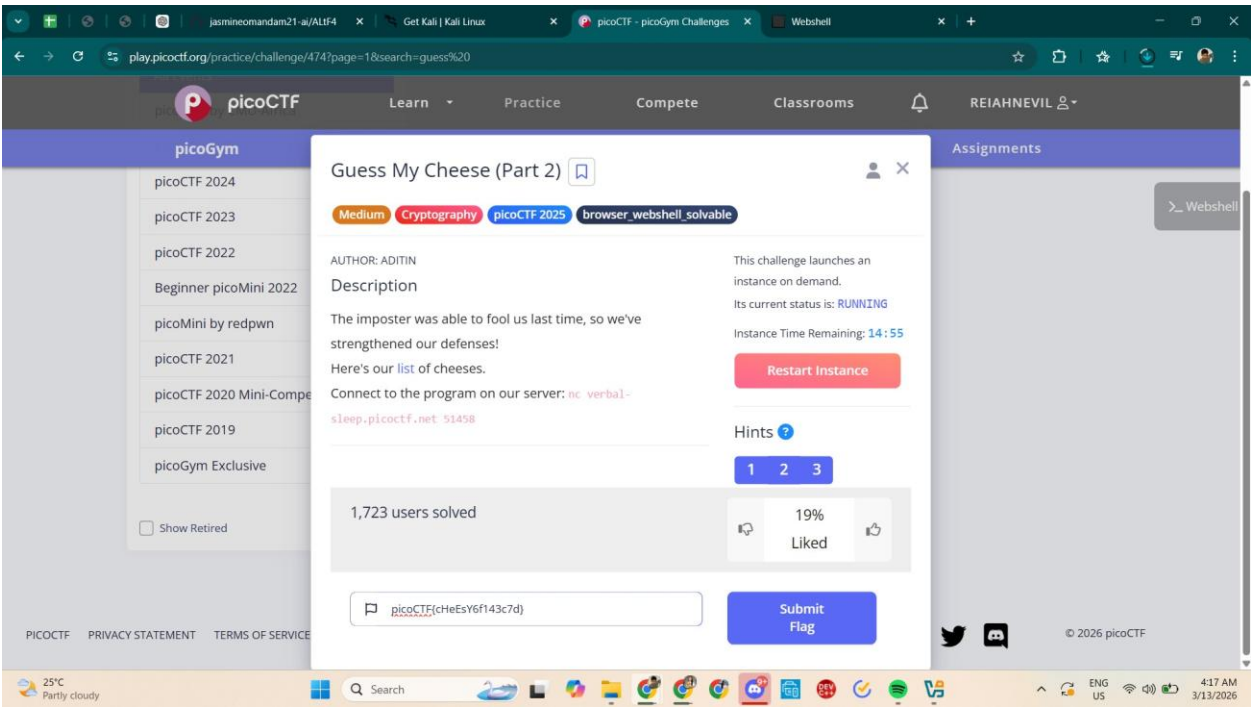


Solving a Symmetric-Cryptography CTF Challenge



CHALLENGE TITLE: **Guess My Cheese Part 2**
(<https://play.picoctf.org/practice/challenge/474?page=1&search=guess%20my%20cheese%20part%202>)
PLATFORM: **picoCTF**
CATEGORY: **Cryptography (Medium)**

Objective:
To solve a cryptography-based Capture the Flag (CTF) challenge involving the SHA-256 hashing algorithm by applying concepts such as hashing, salting, and rainbow tables to recover the original input used to generate a given hash.

Challenge Overview:
The Guess My Cheese Part 2 challenge required identifying the correct cheese name and salt value used to generate a SHA-256 hash. The challenge description hinted that the system applied two hexadecimal nibbles of salt to the cheese name before hashing.

A nibble represents half a byte, equivalent to a single hexadecimal character. Therefore, two nibbles correspond to two hexadecimal characters, giving 256 possible salt combinations. The server would display a SHA-256 hash, and the objective was to determine which cheese from a provided list and which salt value produced that hash.

PROCEDURE:
1. Analyzing the Challenge Hints
The hints suggested several important details:

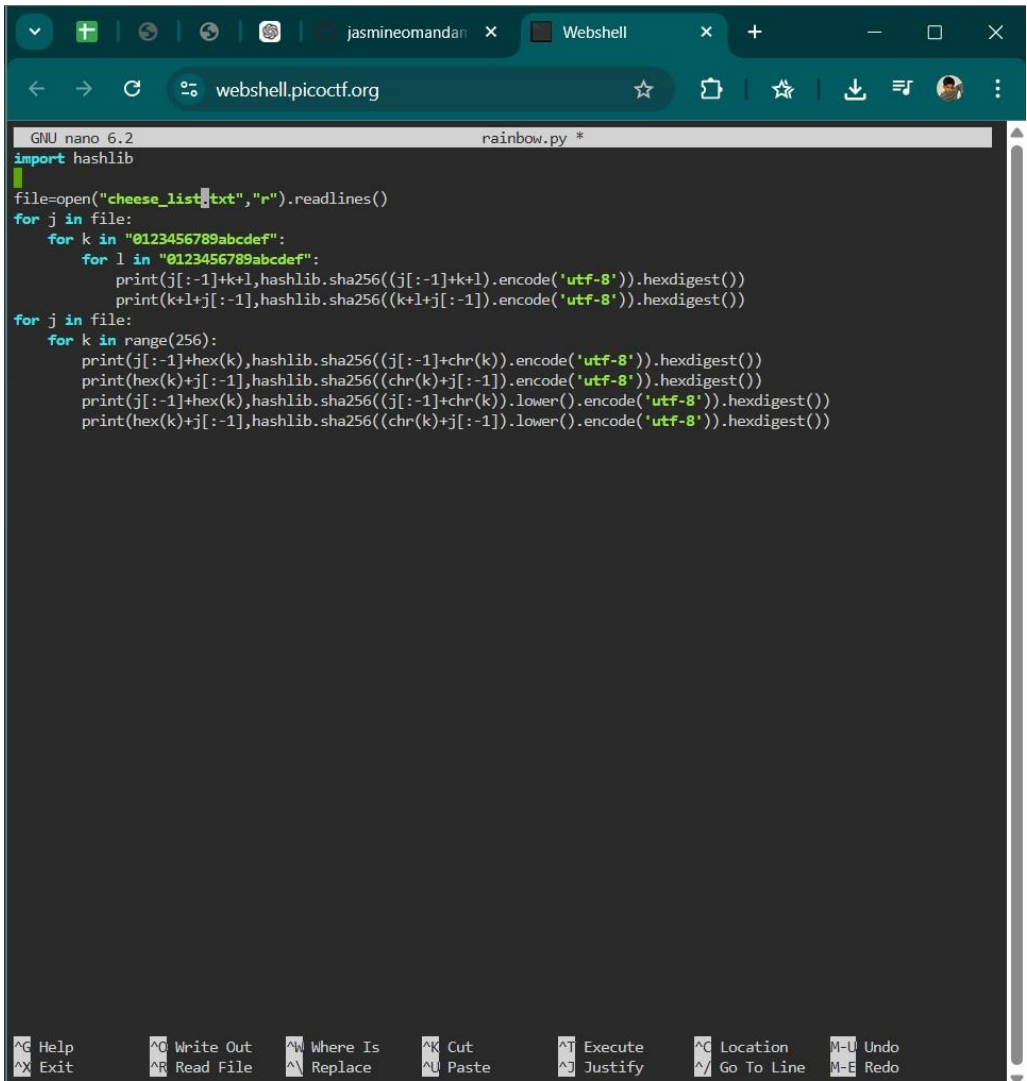
- The hashing algorithm used was **SHA-256**.

- The system added **two hexadecimal characters of salt**.
- The cheese names came from a **known list of cheeses**.
- Rainbow tables could potentially be useful for solving the challenge.

From these hints, it became clear that the most effective approach would be to **precompute hashes for all possible combinations of cheeses and salts**, then compare them with the hash provided by the server.

2. Creating a Rainbow Table

To automate this process, I created a Python script named **rainbow.py**. The script used the **hashlib** library to compute SHA-256 hashes.

A screenshot of a web browser window displaying a webshell interface. The browser's address bar shows 'webshell.picocft.org'. The webshell window has a title bar with 'jasmineomandan' and 'Webshell'. Inside the webshell, a GNU nano 6.2 editor is open, editing a file named 'rainbow.py'. The script content is as follows:

```
import hashlib
file=open("cheese_list.txt","r").readlines()
for j in file:
    for k in "0123456789abcdef":
        for l in "0123456789abcdef":
            print(j[:-1]+k+l,hashlib.sha256((j[:-1]+k+l).encode('utf-8')).hexdigest())
            print(k+l+j[:-1],hashlib.sha256((k+l+j[:-1]).encode('utf-8')).hexdigest())
for j in file:
    for k in range(256):
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).encode('utf-8')).hexdigest())
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).lower().encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).lower().encode('utf-8')).hexdigest())
```

The bottom of the nano editor shows standard keyboard shortcuts like ^G Help, ^O Write Out, ^W Where Is, etc.

- The program performed the following steps:
1. Read a file containing the list of cheeses.
 2. Generate all possible two-character hexadecimal salt combinations.
 3. Append each salt value to each cheese name.
 4. Convert the string to lowercase to match the hashing format used by the server.
 5. Compute the SHA-256 hash.
 6. Store the hash and corresponding cheese-salt combination.

The script essentially generated a **rainbow table**, which maps hashes back to their original inputs.

3. Generating the Rainbow Table

I executed the script with the following command: `python rainbow.py > table`

```
REIAHNEVIL-picocftf@webshell:~$ ls
README.txt  cheese_list.txt  data.enc  data.enc.1  new_caesar.py  rainbow.py  table
REIAHNEVIL-picocftf@webshell:~$ cat rainbow.py
import hashlib

file=open("cheese_list.txt","r").readlines()
for j in file:
    for k in "0123456789abcdef":
        for l in "0123456789abcdef":
            print(j[:-1]+k+l,hashlib.sha256((j[:-1]+k+l).encode('utf-8')).hexdigest())
            print(k+l+j[:-1],hashlib.sha256((k+l+j[:-1]).encode('utf-8')).hexdigest())
for j in file:
    for k in range(256):
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).encode('utf-8')).hexdigest())
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).lower().encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).lower().encode('utf-8')).hexdigest())
REIAHNEVIL-picocftf@webshell:~$ python rainbow.py > table
REIAHNEVIL-picocftf@webshell:~$
```

This redirected the output of the program into a file named `table`, which contained the precomputed hashes and their associated cheese names and salts.

The resulting table allowed me to quickly search for any hash returned by the challenge server.

4. Connecting to the Challenge Server

Next, I connected to the remote challenge server using netcat. The server generated a hash corresponding to a salted cheese name. I copied the hash and searched for it within the rainbow table using a search command. Once the matching hash was located, the corresponding entry revealed the correct cheese name and salt.

```
REIAHNEVIL-picocftf@webshell:~$ nc verbal-sleep.picocftf.net 51458
*****
*** Part 2 ***
*** The Mystery of the CLONED RAT ***
*****

DRAT! The evil Dr. Lacktoes Inn Tolerant's clone was able to guess the cheese last time! I guess simple ciphers aren't good hashing methods. But now I've strengthened my encryption scheme so that now ONLY SQUEEZY can guess it...

Here's my secret cheese -- if you're Squeazy, you'll be able to guess it: Be38e0d31a1ee144b4740a33bba3d00bab7198b22928341bd9150339db594235

Commands: (g)uess my cheese
what would you like to do?
?
[1]: Stopped nc verbal-sleep.picocftf.net 51458
REIAHNEVIL-picocftf@webshell:~$ cat table | grep Be38e0d31a1ee144b4740a33bba3d00bab7198b22928341bd9150339db594235
bash: grep: Be38e0d31a1ee144b4740a33bba3d00bab7198b22928341bd9150339db594235: command not found
REIAHNEVIL-picocftf@webshell:~$ cat table | grep Be38e0d31a1ee144b4740a33bba3d00bab7198b22928341bd9150339db594235
Fondant de Brebis051 Be38e0d31a1ee144b4740a33bba3d00bab7198b22928341bd9150339db594235
Fondant de Brebis071 Be38e0d31a1ee144b4740a33bba3d00bab7198b22928341bd9150339db594235
REIAHNEVIL-picocftf@webshell:~$ fg
nc verbal-sleep.picocftf.net 51458
6

(q\~p)
/ \
= \ t / =
\ \ ) \
( / ) \
m m

SQUEAK SQUEAK SQUEAK

(q\~p)
/ \
= \ t / =
\ \ ) \
( / ) \
m m

Is that you, Squeazy? Are you ready to GUESS...MY...CHEEEEEEESE?
Remember, this is my encrypted cheese: Be38e0d31a1ee144b4740a33bba3d00bab7198b22928341bd9150339db594235
So...what's my cheese?
```

5. Identifying the Correct Answer

The rainbow table search revealed the correct combination. After submitting this combination to the challenge server, the system confirmed the correct answer and returned the flag.

